Tarea 2: Parametrización de superficies

Pesos, bordes y distorsión

CC5513 – Procesamiento Geométrico y Análisis de Formas
Profesor: Iván Sipiran

1. Introducción

Dada una superficie S con topología de disco y un dominio plano Ω, parametrizar S consiste en encontrar un mapa biyectivo $U: S \rightarrow \Omega$. En una malla de triángulos basta con asignar a cada vértice v_i unas coordenadas $u_i \in \mathbb{R}^2$; dentro de cada triángulo el mapa es lineal. Con la parametrización se pueden aplicar texturas, remallar o transferir información entre superficies.

En el **modelo de resortes** se imagina que cada arista es un resorte de rigidez D_{ij} y se busca la posición de equilibrio de la malla aplanada, es decir, las coordenadas que minimizan la energía

$$E(U) = \frac{1}{2}\sum_{i=1}^{n}\sum_{j\in N_i} D_{ij}\|u_i - u_j\|^2 \tag{1}$$

con los vértices del borde fijos en posiciones b_i elegidas de antemano. Como vimos en clase, el mínimo se obtiene resolviendo el sistema lineal

$$LU = 0 \text{ para los vértices interiores}, \quad u_i = b_i \text{ en el borde} \tag{2}$$

donde L es la matriz Laplaciana asociada a los pesos:

$$L_{ii} = \sum_{j\in N_i} D_{ij}, \quad L_{ij} = -D_{ij} \text{ si } j \in N_i, \quad L_{ij} = 0 \text{ en otro caso} \tag{3}$$

La elección de los pesos D_{ij} y de la curva donde se fija el borde determinan la parametrización. En esta tarea se usan tres familias de pesos (ver la Figura 1 para la notación de los ángulos):

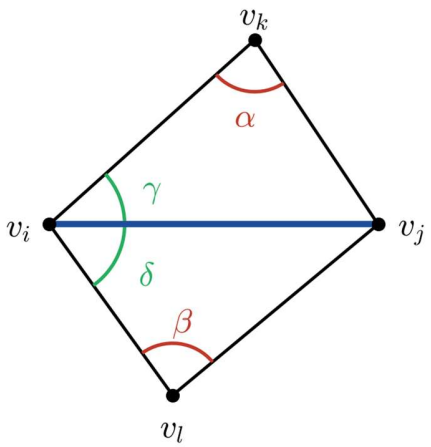


Figura 1: Arista (v_i, v_j) y sus dos triángulos vecinos. α y β son los ángulos *opuestos* a la arista; γ y δ son los ángulos *en* v_i.

- **Uniformes** (Tutte): todas las aristas tienen la misma rigidez.

$$D_{ij} = 1 \tag{4}$$

- **Armónicos** (cotangentes; Pinkall y Polthier): dependen de los ángulos opuestos a la arista.

$$D_{ij} = \frac{1}{2}(cot\,\alpha + cot\,\beta) \tag{5}$$

- **Valor medio** (*mean value*; Floater): dependen de los ángulos en v_i y de la longitud de la arista. Notar que en general $D_{ij} \neq D_{ji}$.

$$D_{ij} = \frac{tan(\gamma/2)+tan(\delta/2)}{\|v_i - v_j\|} \tag{6}$$

> **Teorema (Tutte 1963; Floater 1997).** Si todos los pesos de las aristas interiores son positivos y el borde se lleva de forma ordenada a un polígono **convexo**, la parametrización resultante es biyectiva: ningún triángulo queda invertido. Gran parte de esta tarea consiste en observar qué ocurre cuando alguna de estas hipótesis no se cumple.

En esta tarea se implementan y comparan estos pesos, se estudia el efecto de la forma del borde y se analizan, a partir de una implementación que se entrega completa, un método de **borde libre** (LSCM) y un conjunto de **medidas cuantitativas de distorsión** que complementan la inspección visual de las texturas.

2. Código base

Se entrega el paquete `tarea3_param` con el código base en Python (NumPy, SciPy y Polyscope) y las tres mallas de prueba en `data/`: **Nefertiti** (299 vértices), **Bunny** (1.573) y **Lucy** (14.156). Las tres tienen topología de disco (un único borde).

Archivo	Contenido	¿Modificar?
`mesh_io.py`, `mesh_utils.py`	Lectura OFF, borde, ángulos, áreas, coordenadas locales, malla plana de prueba	No
`laplacians.py`	Pesos uniformes (dado); pesos cotangentes y de valor medio	**Parte 1**
`boundary.py`	Espaciado del borde y círculo (dados); borde poligonal	**Parte 2**
`solvers.py`	Solver de borde fijo, LSCM y energía conforme (dados)	No; se analiza en la **Parte 3**
`distortion.py`	Jacobianos y medidas de distorsión (dados)	No; se analiza en la **Parte 4**
`param.py`, `viewer.py`, `run_tarea3.py`	Integración, visor Polyscope y programa principal	No
`tests.py`	Pruebas automáticas de cada parte	No (puede agregar)

Las funciones que debe completar lanzan `NotImplementedError` y su docstring describe la fórmula, el formato de entrada y salida, y una pista de implementación. Mientras una parte no esté implementada, el visor y las pruebas la muestran como `PENDIENTE` sin interrumpir el resto. Los comandos principales son:

```
# visor interactivo
python run_tarea3.py --mesh data/nefertiti.off
# elegir método y borde desde la línea de comandos
python run_tarea3.py --mesh data/bunny.off --method meanvalue --boundary square
# tabla de métricas de todas las combinaciones
python run_tarea3.py --mesh data/lucy.off --batch --csv lucy.csv
# capturas fig_3d.png y fig_uv.png, sin abrir ventana
python run_tarea3.py --mesh data/bunny.off --method lscm --color conformal --shot fig
# pruebas automáticas
python tests.py
```

El visor muestra la superficie con la textura de tablero y, a su lado, la malla aplanada en el plano (u, v) con su borde en rojo. Desde el panel se cambian el método (`uniform`, `harmonic`, `meanvalue`, `lscm`), la forma del borde (`circle`, `square`, `star`), el espaciado (`arc`, `uniform`) y lo que se colorea (`checker`, `conformal`, `area`, `flips`). La opción `--shot` guarda las imágenes sin abrir la ventana, lo que es útil para el informe. La Figura 2 muestra lo que se obtiene con el código tal como se entrega.

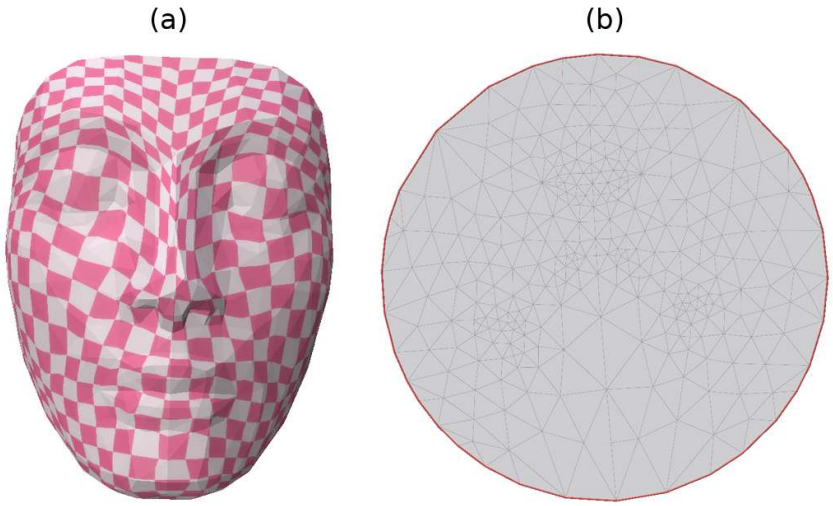


Figura 2: Resultado del código base en Nefertiti (pesos uniformes, borde circular): (a) textura de tablero sobre la superficie; (b) malla en el dominio (u, v).

El solver de borde fijo (`solve_fixed_boundary`, ya implementado), en lugar de reemplazar las filas de L correspondientes al borde, separa los vértices interiores (I) de los de borde (B) y resuelve el sistema reducido, que tiene una incógnita por vértice interior:

$$L_{II} U_I = -L_{IB} U_B \tag{7}$$

3. Enunciado

La tarea tiene cinco partes. Las Partes 1 y 2 son de implementación. En las Partes 3 y 4 el código de LSCM y de las medidas de distorsión se entrega completo: el trabajo consiste en entender cómo funciona, verificarlo con experimentos diseñados por usted y explicar su comportamiento. La Parte 5 es un análisis comparativo de los métodos sobre las tres mallas. Todas las preguntas se responden en el informe con figuras y números.

Parte 1. Pesos armónicos y de valor medio (1,25 pts)

Implemente `cotangent_weights(V, F)` y `mean_value_weights(V, F)` en `laplacians.py`. Ambas devuelven una matriz dispersa W de $n \times n$ con $W[i,j] = D_{ij}$; el Laplaciano se arma con `laplacian_from_weights`, que calcula $L = diag(d) - W$, donde d es el vector de sumas de las filas de W. Observaciones:

- La implementación debe estar **vectorizada**: se permite un ciclo sobre las 3 esquinas de los triángulos, pero no sobre los vértices ni sobre las caras. Use `corner_angles` y la función auxiliar `_sparse_from_triplets`, que suma las entradas repetidas. Con Lucy (28.048 triángulos) cada matriz debería calcularse en menos de un segundo.
- En una arista de borde hay un solo triángulo y por lo tanto un solo término de cada suma.
- No corrija los pesos cotangentes negativos: se analizan en las preguntas.

Prueba de reproducción lineal. `tests.py` construye una malla plana con vértices interiores perturbados al azar (Figura 3), fija el borde en su posición original y resuelve. Un método con *precisión lineal* debe recuperar exactamente las coordenadas originales de todos los vértices. Sus pesos deben pasar las pruebas 1a y 1b.

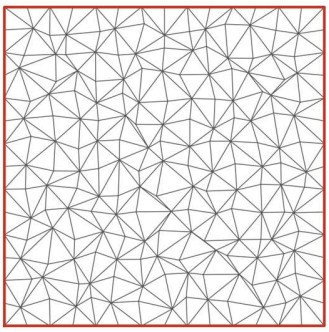


Figura 3: Malla plana de prueba (196 celdas, vértices interiores perturbados) con su borde en rojo.

Preguntas.

1.1 Reporte el número de pesos cotangentes negativos de cada malla (columna `neg_w` de `--batch`). ¿Qué condición geométrica sobre α y β produce $D_{ij} < 0$? ¿Se cumplen entonces las hipótesis del teorema de Tutte–Floater? Busque una combinación de malla y borde circular en que aparezca algún triángulo invertido con pesos armónicos y muéstrelo con el modo de color `flips`.

1.2 Demuestre que los pesos de valor medio son siempre positivos. ¿Qué garantiza esto?

1.3 Explique por qué los pesos uniformes no pasan la prueba de reproducción lineal y los otros dos sí. ¿Qué consecuencia práctica tiene esto en regiones de la superficie que son casi planas?

Parte 2. Bordes poligonales (1,0 pt)

La función `boundary_parameter` (dada) asigna a cada vértice del borde un parámetro t_i en el intervalo [0, 1), ya sea proporcional a la longitud de arco del borde 3D (`arc`) o uniforme, $t_i = i/n_b$ (`uniform`). El borde circular simplemente toma $b_i = (cos\ 2\pi t_i\ , sin\ 2\pi t_i)$.

Implemente `polygon_points(t, corners)` en `boundary.py`, que recorre el perímetro de un polígono a **velocidad constante**: $t = 0$ corresponde al primer vértice del polígono y $t = s/P$ al punto que está a distancia s de él medida a lo largo del perímetro P. Con esto quedan disponibles los bordes `square` y `star` (Figura 4). La estrella **no es convexa**.

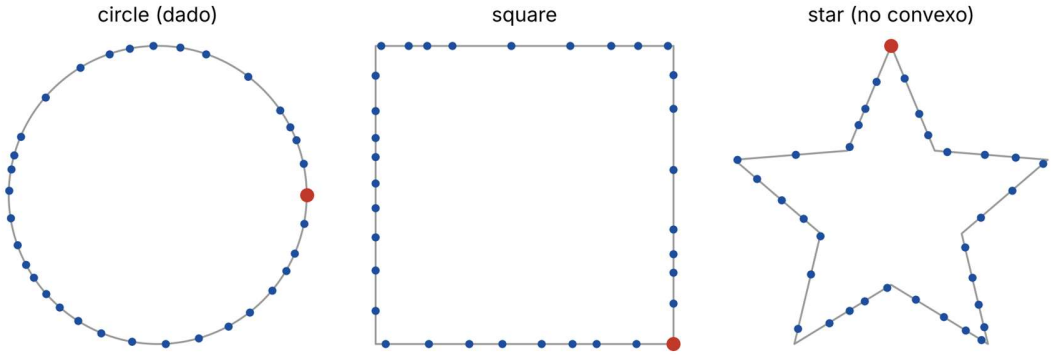


Figura 4: Posición de los 34 vértices del borde de Nefertiti en las tres curvas destino, con espaciado por longitud de arco. El punto rojo es el primer vértice del borde.

Calcule las parametrizaciones de las tres mallas con las tres formas de borde y los dos espaciados, usando los tres tipos de pesos (la opción `--batch` lo hace automáticamente).

Preguntas.

2.1 Con el borde cuadrado aparecen triángulos degenerados (columna `degen`) en Bunny, incluso con pesos positivos. Identifique dónde están y explique por qué se producen. ¿Contradice esto el teorema? Proponga una forma de evitarlos.

2.2 Con la estrella aparecen triángulos invertidos para todos los pesos. Relacione esto con las hipótesis del teorema y muestre dónde se concentran los triángulos invertidos.

2.3 Compare los espaciados `arc` y `uniform` en términos de distorsión conforme y de área. ¿En qué malla importa más la diferencia y por qué? (Pista: mire la distribución de longitudes de las aristas del borde.)

Parte 3. Análisis de la parametrización con borde libre: LSCM (1,25 pts)

Fijar el borde de antemano introduce distorsión cerca de él. *Least Squares Conformal Maps* (LSCM; Lévy et al. 2002) deja el borde libre y busca el mapa **lo más conforme posible**. El código base lo implementa con la formulación de Mullen et al. (2008), que reutiliza el Laplaciano cotangente de la Parte 1 (por eso LSCM funciona una vez completada la Parte 1a). La energía conforme es la diferencia entre la energía de Dirichlet y el área de la parametrización:

$$E_C(u,v) = E_D(u,v) - A(u,v) = \frac{1}{2}(u^T L u + v^T L v) - u^T K v \tag{8}$$

donde L es el Laplaciano cotangente y K es la matriz que, recorriendo el borde en el sentido de las caras, calcula el área con signo:

$$A(u,v) = \frac{1}{2}\sum_{(i\to j)\in\partial S}(u_i v_j - u_j v_i), \quad K_{ij} = \frac{1}{2},\ K_{ji} = -\frac{1}{2} \tag{9}$$

Se cumple $E_C \geq 0$, con igualdad solo si el mapa es conforme. Si se apilan las incógnitas en $x = [u;\ v]$, la energía es $E_C = x^T Q x/2$, donde Q es una matriz de $2n \times 2n$ formada por los bloques $Q_{uu} = Q_{vv} = L$, $Q_{uv} = -K$ y $Q_{vu} = K$. Para obtener una solución se fijan dos vértices del borde (*pins*): $p_0 \to (0,0)$ y $p_1 \to (d,0)$. Si f son las variables libres y p las fijas, el mínimo resuelve $Q_{ff} x_f = -Q_{fp} x_p$.

Las funciones relevantes están en `solvers.py`: `boundary_area_matrix`, `lscm`, `choose_pins` (elige dos vértices del borde lejanos entre sí) y `conformal_energy`, que evalúa E_D, A, E_C y el cociente E_D/A para **cualquier** parametrización (como el borde se recorre en el sentido de las caras, $A > 0$ si la parametrización preserva la orientación). Ese cociente no depende de la escala, vale 1 solo para un mapa conforme y aparece en la columna `ED/|A|` de `--batch`. Los *pins* se pueden elegir con `--pins A B` (posiciones a lo largo del borde), desde el panel del visor (donde se marcan en azul) o con el argumento `pins` de `parameterize`.

Para los experimentos puede escribir sus propios scripts usando los módulos del código base; inclúyalos en la entrega (ver Sección 4).

Preguntas.

3.1 **Lectura del código.** Explique paso a paso `boundary_area_matrix` y `lscm`, relacionando cada bloque de código con las ecuaciones (8) y (9). Deduzca, derivando E_C respecto de x_f, por qué el sistema $Q_{ff} x_f = -Q_{fp} x_p$ entrega el mínimo. ¿Qué ocurriría si el borde se recorriera en el sentido contrario al de las caras?

3.2 **¿Por qué dos pins?** Calcule los valores propios más pequeños de Q para Nefertiti (la matriz es de 598×598, así que puede usar `np.linalg.eigvalsh(Q.toarray())`). ¿Cuántos son cero y a qué movimientos de la parametrización corresponden? Use este resultado para explicar qué pasaría si se fijara un solo vértice y por qué hacen falta dos.

3.3 **Dependencia de los pins.** Para cada malla, deje fijo el primer *pin* en la posición 0 del borde y mueva el segundo a lo largo de todo el borde. Grafique E_D/A, `conf_med` y `area_mean` en función de la posición del segundo *pin*, y mida cuánto difiere cada resultado del obtenido con los *pins* automáticos **después de alinearlos con la mejor semejanza** (escriba una función de alineamiento de Procrustes). ¿El resultado cambia más allá de una semejanza? ¿Qué elección de *pins* recomienda y por qué?

Parte 4. Análisis de las medidas de distorsión (1,0 pt)

Dentro de cada triángulo la parametrización es una función afín. Si q_0, q_1, q_2 son las coordenadas del triángulo 3D en un sistema local 2D isométrico (`local_triangle_coords`) y u_0, u_1, u_2 sus coordenadas en el plano, la parte lineal del mapa es la matriz J de 2×2 que lleva las aristas del triángulo 3D a las del triángulo plano:

$$J(q_1 - q_0) = u_1 - u_0, \quad J(q_2 - q_0) = u_2 - u_0 \tag{10}$$

Sus valores singulares $\sigma_1 \geq \sigma_2$ son los estiramientos máximo y mínimo del triángulo. A partir de ellos se definen la **distorsión conforme** σ_1/σ_2 (vale 1 si se preservan los ángulos), la **distorsión de área** $log_2(\sigma_1 \sigma_2)$ (vale 0 si se preserva el área) y los **triángulos invertidos**, que son aquellos con $det\, J < 0$.

Todo esto ya está implementado en `distortion.py` (`jacobians`, `face_distortion` y `distortion_report`), así que esta parte se puede desarrollar desde el inicio con los pesos uniformes. `distortion_report` normaliza la escala global de (u, v) y entrega, por cara y como resumen, la mediana y el percentil 95 de σ_1/σ_2 (`conf_med`, `conf_p95`), la media ponderada del valor absoluto de $log_2(\sigma_1\sigma_2)$ (`area_mean`), el número de triángulos invertidos (`flips`) y el de degenerados (`degen`). Estas medidas alimentan los modos de color `conformal`, `area` y `flips` del visor (Figura 5) y la tabla de `--batch`.

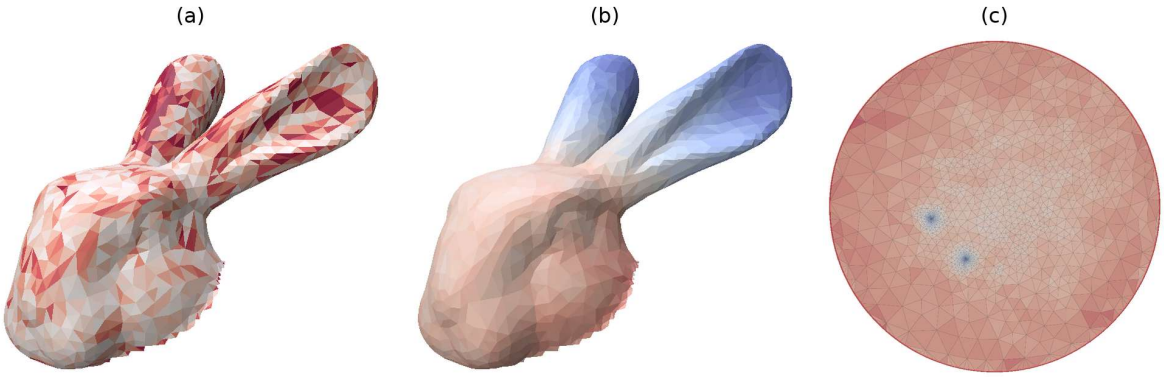


Figura 5: Visualización de la distorsión en Bunny con pesos uniformes y borde circular: (a) distorsión conforme σ_1/σ_2 (blanco = 1); (b) distorsión de área sobre la superficie (azul = comprimido, rojo = expandido); (c) la misma medida en el plano. Las orejas quedan fuertemente comprimidas.

Preguntas.

4.1 **Lectura del código.** Explique `jacobians` y `face_distortion`. ¿Por qué se trabaja en un sistema local 2D por triángulo y no directamente con las coordenadas 3D? Muestre que J depende del sistema local elegido pero que σ_1, σ_2 y el signo de $det\, J$ no. ¿Qué relación hay entre $det\, J$, el cociente de áreas y la orientación del triángulo?

4.2 **Experimentos controlados.** Use como parametrización de la malla plana de prueba transformaciones conocidas: (a) rotación con escala uniforme; (b) escala anisótropa $(x, y) \to (3x, y)$; (c) cizalle $(x, y) \to (x + 0{,}5y, y)$; (d) reflexión $(x, y) \to (-x, y)$; (e) el mapa conforme no lineal $z \to z^2$ con $z = (x + 1) + iy$. **Antes de calcular**, prediga para cada caso los valores de `conf_med`, `area_mean` y `flips`; luego compárelos con lo que entrega `distortion_report` y explique las diferencias. Muestre cómo se ve la textura de tablero en (b), (c) y (e).

Parte 5. Análisis comparativo (1,5 pts)

Para cada malla, muestre los cuatro métodos (`uniform`, `harmonic` y `meanvalue` con borde circular y espaciado `arc`, más `lscm`) con la textura de tablero sobre la superficie, el dominio (u, v) y al menos un mapa de distorsión. Use un tamaño de tablero que permita ver los detalles (opción `--checker`). Responda:

5.1 Ordene los cuatro métodos según su distorsión conforme y según su distorsión de área en cada malla. ¿Hay algún método que gane en ambas? Explique por qué en general no se pueden preservar a la vez ángulos y áreas.

5.2 Las orejas de Bunny y las alas y la antorcha de Lucy quedan muy comprimidas con todos los métodos (valores de `area_mean` entre 4 y 8 significan que un triángulo típico cambia su área en un

factor de 2^4 a 2^8). Explique por qué las protuberancias alejadas del borde se comprimen de esta manera, incluso con LSCM, cuyo borde es libre. Usando lo observado en la pregunta 4.5, proponga (sin implementar) una estrategia para resolverlo.

5.3 Si tuviera que aplicar una fotografía como textura sobre Nefertiti, ¿qué método y qué borde elegiría? Justifique con números e imágenes.

5.4 Compare los tiempos de cálculo de los métodos en Lucy. ¿Qué parte domina el costo?

Bonus (hasta +1,0 pt; la nota máxima es 7,0)

- **B1 (+0,5).** Modifique el mapeo al cuadrado para eliminar los triángulos degenerados de la pregunta 2.1 y muestre `degen = 0` en las tres mallas.
- **B2 (+0,5).** Aplique una imagen real (por ejemplo, una fotografía) como textura usando la parametrización, en lugar del tablero.

4. Entregables

- **Código:** la carpeta completa del proyecto con los archivos modificados (`laplacians.py` y `boundary.py`). Debe ejecutarse con `python tests.py` y `python run_tarea3.py` sin modificaciones. **No cambie las firmas de las funciones**, porque la corrección ejecuta pruebas adicionales sobre ellas.
- **Informe en PDF** (máximo 5 páginas, sin contar anexos) con una breve descripción de la implementación, las figuras y tablas pedidas y las respuestas a todas las preguntas, numeradas como en este enunciado (1.1, 1.2, …).
- Solo se pueden usar NumPy, SciPy, Polyscope y Matplotlib. No se permite usar bibliotecas que calculen parametrizaciones o Laplacianos (libigl, pygeodesic, igl, etc.).

5. Evaluación

Parte	Se evalúa	Puntaje
1. Pesos	Implementación correcta y vectorizada; pruebas 1a y 1b; preguntas 1.1–1.3	1,25
2. Bordes	`polygon_points`; experimentos con los tres bordes; preguntas 2.1–2.3	1,0
3. Análisis de LSCM	Explicación del código; experimentos de núcleo de Q, *pins.*	1,25
4. Análisis de distorsión	Explicación del código; experimentos controlados.	1,0
5. Análisis	Figuras de calidad; respuestas 5.1–5.4 fundamentadas con números e imágenes	1,5
Base		1,0
Total		**7,0**
Bonus	B1, B2 (la nota final no supera 7,0)	+1,0

Que todas las pruebas automáticas pasen es condición necesaria, pero no suficiente: también se revisa la calidad del código, la reproducibilidad de los experimentos y la fundamentación de las respuestas.

Referencias

[1] M. Botsch, L. Kobbelt, M. Pauly, P. Alliez, B. Lévy. *Polygon Mesh Processing*, cap. 5. AK Peters, 2010.

[2] M. S. Floater. Parametrization and smooth approximation of surface triangulations. *Computer Aided Geometric Design* 14(3), 1997.

[3] M. S. Floater. Mean value coordinates. *Computer Aided Geometric Design* 20(1), 2003.

[4] M. S. Floater, K. Hormann. Surface parameterization: a tutorial and survey. En *Advances in Multiresolution for Geometric Modelling*, Springer, 2005.

[5] B. Lévy, S. Petitjean, N. Ray, J. Maillot. Least squares conformal maps for automatic texture atlas generation. *ACM Transactions on Graphics (SIGGRAPH)* 21(3), 2002.

[6] P. Mullen, Y. Tong, P. Alliez, M. Desbrun. Spectral conformal parameterization. *Computer Graphics Forum (SGP)* 27(5), 2008.

[7] U. Pinkall, K. Polthier. Computing discrete minimal surfaces and their conjugates. *Experimental Mathematics* 2(1), 1993.

[8] W. T. Tutte. How to draw a graph. *Proceedings of the London Mathematical Society* 13, 1963.